

LangChain 表达式语言 (LCEL) 入门与实战

彭靖田

Google Developer Experts
LangChain Developer



目录

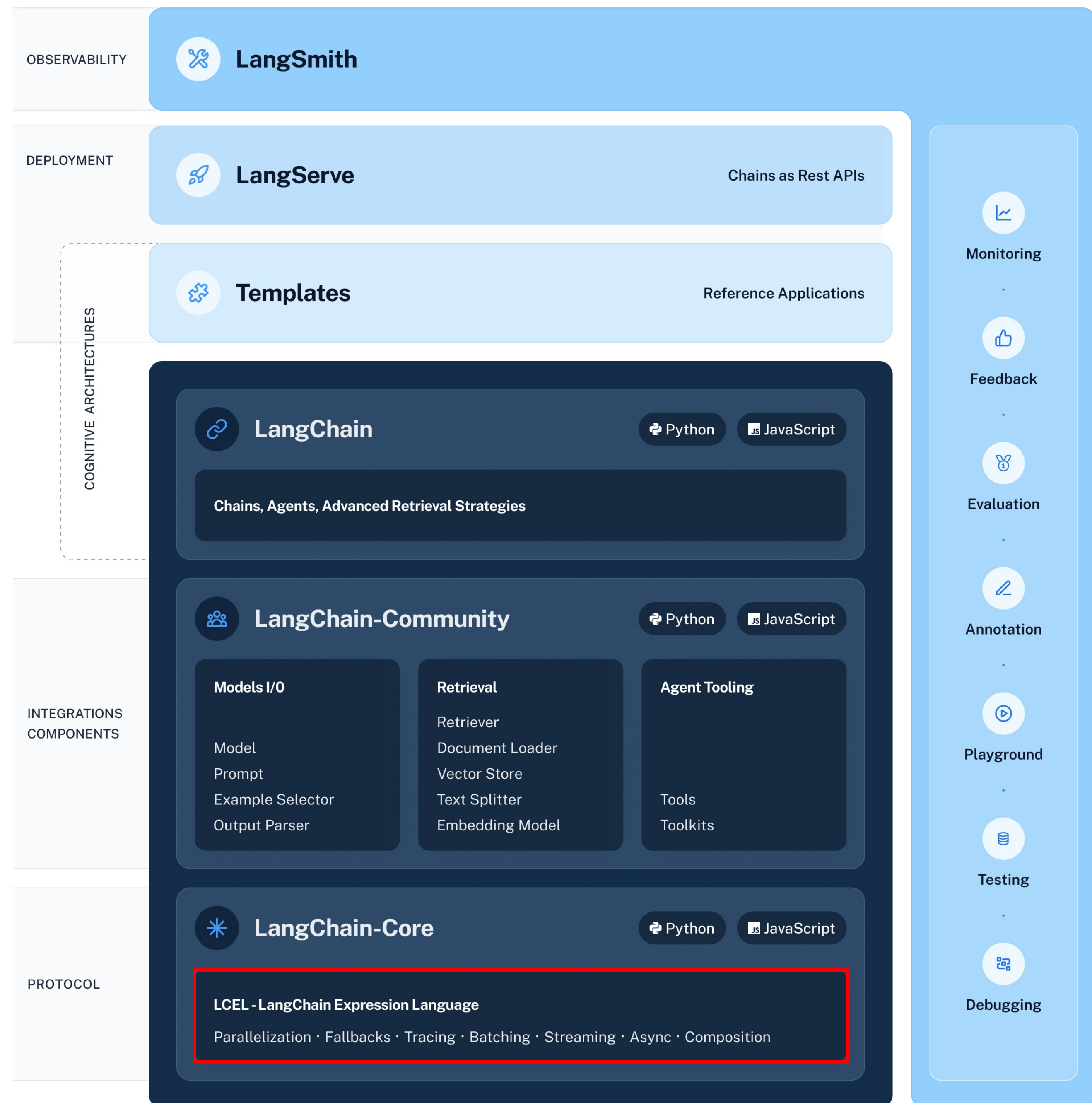
- LangChain Expression Language (LCEL) 快速入门
 - LCEL 是什么
 - LCEL 设计理念与优势
 - LCEL Runnable 协议设计与使用
 - LCEL vs LangChain 古典语法
 - LCEL 进阶使用：整合复杂逻辑的多链
- 实战 LCEL 开发与应用
 - 最简易示例：Prompt + Chat Model + Output Parser
 - 检索增强生成示例：RAG
 - AI辩论示例：Multiple Chains via Runnables

LangChain Expression Language 是什么？

LangChain

技术生态与社区

- LangChain: LLMs 应用开发框架
- LangChain-Community: 第三方集成
- LangChain-Core: LCEL 等协议
- LangChain Templates: 开箱即用APP示例
- LangServe: Chains 生产部署 (REST API)
- LangSmith: 一站式开发者平台



LCEL是一种声明式的链式组合语言，能够从简单的 `Prompt+LLM` 链到复杂的链（运行100多个步骤的生产环境）。

其设计初衷是支持将原型快速投入生产，而无需更改代码。

LCEL使得从基本组件构建复杂链变得容易，并支持流式处理、并行处理和日志记录等开箱即用的功能。

组件

我们已学习的组件包括以下模块：

Model I/O:

这包括提示管理，提示优化，用于聊天模型和LLM的通用接口，以及处理模型输出的常见实用工具。

Retrieval:

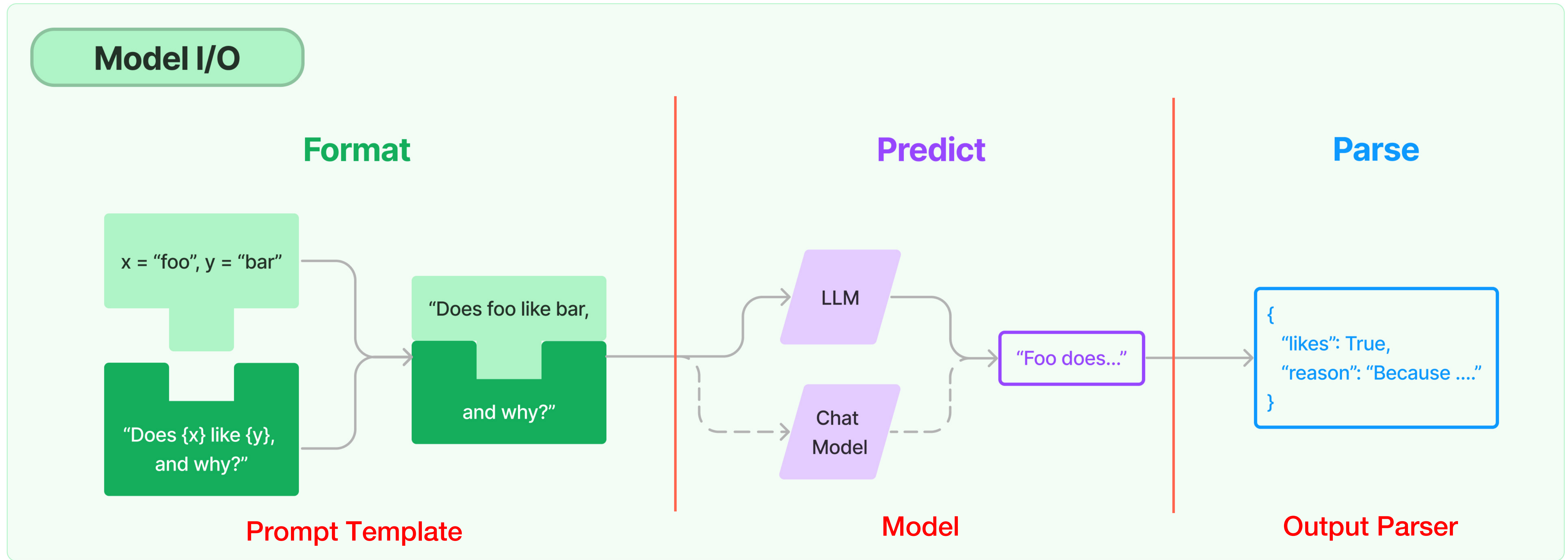
检索增强生成涉及从各种来源加载数据，准备数据，然后在生成步骤中检索数据以供使用。

Agents:

Agents 允许LLM自主完成任务。Agents会决定采取哪些行动，然后执行该行动，并观察结果，并重复此过程直到任务完成。LangChain为代理提供了标准接口、可选择的代理列表以及端到端代理示例。

LCEL 设计理念与优势

温故：标准化的大模型抽象 Model I/O



竖线符号类似于**Unix**管道操作符，它将不同的组件链接在一起，将一个组件的输出作为下一个组件的输入。

```
[1]: from langchain_openai import ChatOpenAI
      from langchain_core.output_parsers import StrOutputParser
      from langchain_core.prompts import ChatPromptTemplate

      # 初始化 ChatOpenAI 模型, 指定使用的模型为 'gpt-3.5-turbo-0125'
      model = ChatOpenAI(model="gpt-3.5-turbo-0125")

      # 创建一个聊天提示模板, 设置模板内容为"讲个关于 {topic} 的笑话吧"
      prompt = ChatPromptTemplate.from_template("讲个关于 {topic} 的笑话吧")

      # 初始化输出解析器, 用于将模型的输出转换为字符串
      output_parser = StrOutputParser()

      # 构建一个处理链, 先通过提示模板生成完整的输入, 然后通过模型处理, 最后解析输出
      chain = prompt | model | output_parser

      # 调用处理链, 传入主题为"程序员", 生成关于程序员的笑话
      chain.invoke({"topic": "程序员"})
```

```
[1]: '为什么程序员总是喜欢用“Ctrl+C”和“Ctrl+V”呢? \n\n因为他们喜欢复制粘贴! '
```

核心概念：invoke 方法

Langchain `invoke` 方法是 LCEL 设计中的重要方法，可以帮助开发者更高效地处理复杂任务，结合语言模型进行系统构建，实现不同数据源和API的接口对接。

基础概念

- 在Langchain中，`invoke`方法是所有LangChain表达式语言（LCEL）对象的通用同步调用方法。通过`invoke`方法，开发者可以直接调用LLM或ChatModel，简化了调用流程，提高了开发效率。
- 与其他链式调用方法相比，`invoke`方法更加灵活便捷，可以直接对输入进行调用，而不需要额外的链式操作。相对于`batch`和`stream`等异步方法，`invoke`方法更适用于单一操作的执行。

使用方式

- 单次调用：通过`invoke`方法，开发者可以快速对单一操作进行执行，例如转换ChatMessage为Python字符串等简单操作，提升了代码的可读性和整洁度。
- 复杂协作：Langchain的核心理念就是将语言模型作为协作工具，`invoke`方法可以很好地实现开发者与语言模型的高效互动，构建出处理复杂任务的系统，并对接不同的数据源和API接口。

LCEL Runnable 协议设计与使用

Runnable Interface 介绍与使用

为了尽可能简化创建自定义链的过程，Langchain 实现了一个 **Runnable** 协议。

许多 LangChain 组件都实现了 Runnable 协议，包括 chat models, LLMs, output parsers, retrievers, prompt templates 等等。此外，还有几个用于处理可运行对象的**有用原语**。

Runnable 是一个标准接口，包括：

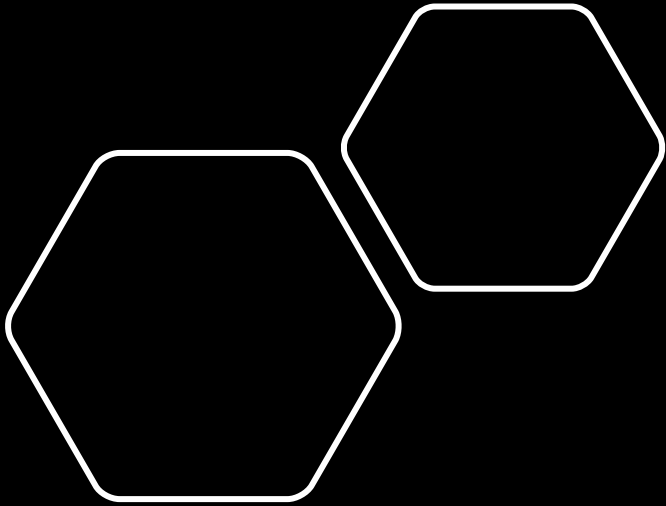
- stream：流式返回生成内容（chunk）
- invoke：对输入调用该链
- batch：对输入列表调用该链

不同组件的输入和输出类型有所差异：

组件	输入类型	输出类型
Prompt	Dictionary	PromptValue
ChatModel	Single string, list of chat messages or a PromptValue	ChatMessage
LLM	Single string, list of chat messages or a PromptValue	String
OutputParser	The output of an LLM or ChatModel	Depends on the parser
Retriever	Single string	List of Documents
Tool	Single string or dictionary, depending on the tool	Depends on the tool

所有 Runnable 对象都显式描述输入和输出 Schema，以检查输入和输出格式：

- input_schema：从Runnable的结构自动生成的输入Pydantic模型
- output_schema：从Runnable的结构自动生成的输出Pydantic模型



Input Schema

为了演示如何使用，下面我们创建一个超级简单的PromptTemplate + ChatModel链。

```
[1]: from langchain_openai import ChatOpenAI
      from langchain_core.prompts import ChatPromptTemplate

      model = ChatOpenAI(model="gpt-3.5-turbo-0125")
      prompt = ChatPromptTemplate.from_template("讲个关于 {topic} 的笑话吧")
      chain = prompt | model
```

schema 方法

一个描述 Runnable 接受的输入的说明。这是根据任何 Runnable 结构动态生成的 Pydantic 模型。您可以调用 `.schema()` 来获取 `JSONSchema` 表示。

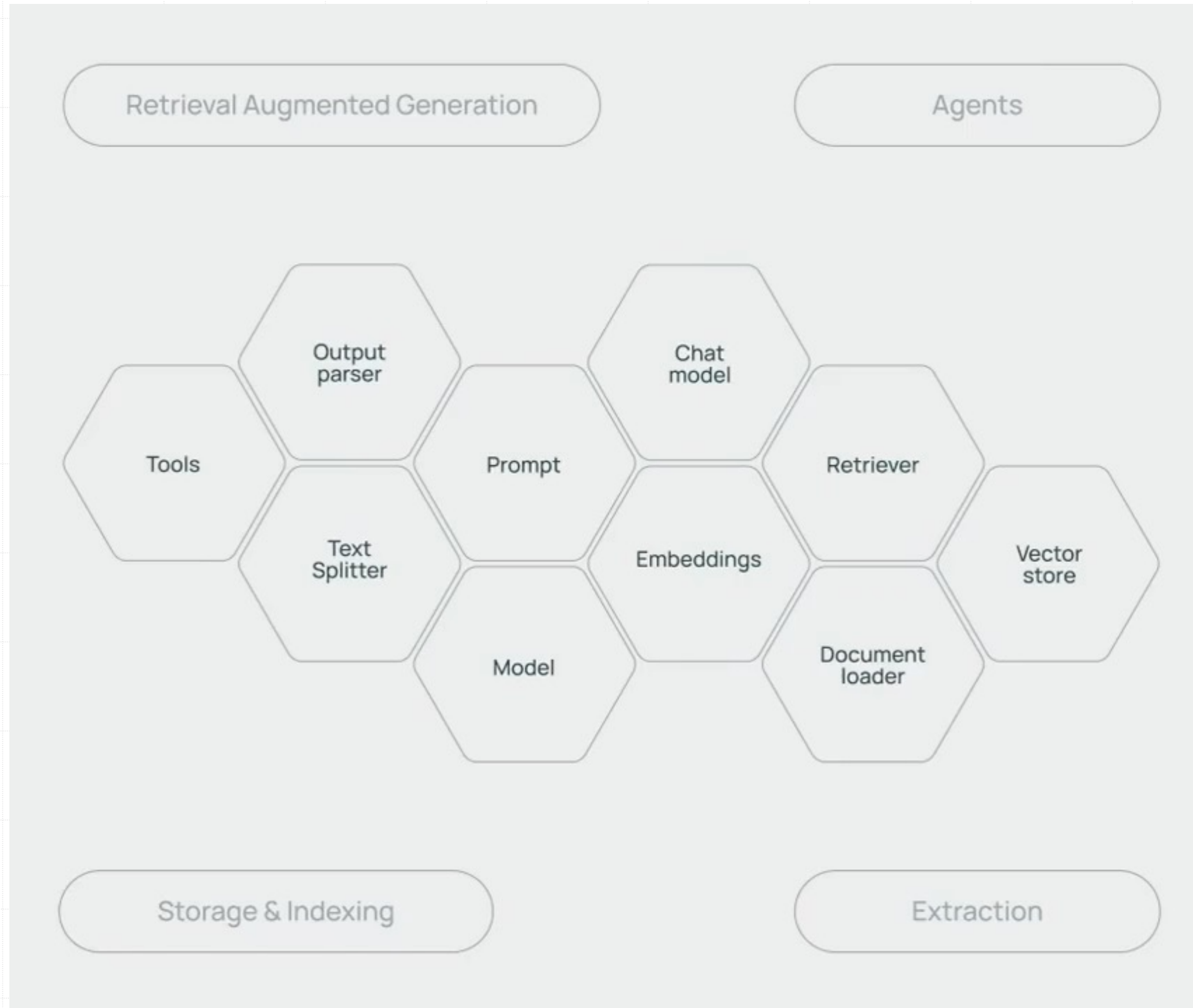
```
• [2]: # 查看 Chain 的输入类型
      chain.input_schema.schema()
```

```
[2]: {'title': 'PromptInput',
      'type': 'object',
      'properties': {'topic': {'title': 'Topic', 'type': 'string'}}}
```

```
• [4]: # 查看 Prompt 的输入类型 (Chain的输入从 Prompt 开始, 因此输入类型一致)
      prompt.input_schema.schema()
```

```
[4]: {'title': 'PromptInput',
      'type': 'object',
      'properties': {'topic': {'title': 'Topic', 'type': 'string'}}}
```

使用 LCEL 串联 LangChain 基础组件构造复杂应用



LCEL vs LangChain 古典语法

In the simplest case, we just want to pass in a topic string and get back a joke string:

Without LCEL

```
from typing import List

import openai

prompt_template = "Tell me a short joke about {topic}"
client = openai.OpenAI()

def call_chat_model(messages: List[dict]) -> str:
    response = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=messages,
    )
    return response.choices[0].message.content

def invoke_chain(topic: str) -> str:
    prompt_value = prompt_template.format(topic=topic)
    messages = [{"role": "user", "content": prompt_value}]
    return call_chat_model(messages)

invoke_chain("ice cream")
```

LCEL

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

prompt = ChatPromptTemplate.from_template(
    "Tell me a short joke about {topic}"
)
output_parser = StrOutputParser()
model = ChatOpenAI(model="gpt-3.5-turbo")
chain = (
    {"topic": RunnablePassthrough()}
    | prompt
    | model
    | output_parser
)

chain.invoke("ice cream")
```

If we want to stream results instead, we'll need to change our function:

Without LCEL

```
from typing import Iterator

def stream_chat_model(messages: List[dict]) -> Iterator[str]:
    stream = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=messages,
        stream=True,
    )
    for response in stream:
        content = response.choices[0].delta.content
        if content is not None:
            yield content

def stream_chain(topic: str) -> Iterator[str]:
    prompt_value = prompt.format(topic=topic)
    return stream_chat_model([{"role": "user", "content": prompt_value}])

for chunk in stream_chain("ice cream"):
    print(chunk, end="", flush=True)
```

LCEL

```
for chunk in chain.stream("ice cream"):
    print(chunk, end="", flush=True)
```

If we want to run on a batch of inputs in parallel, we'll again need a new function:

Without LCEL

```
from concurrent.futures import ThreadPoolExecutor

def batch_chain(topics: list) -> list:
    with ThreadPoolExecutor(max_workers=5) as executor:
        return list(executor.map(invoker_chain, topics))

batch_chain(["ice cream", "spaghetti", "dumplings"])
```

LCEL

```
chain.batch(["ice cream", "spaghetti", "dumplings"])
```

Async

If we need an asynchronous version:

Without LCEL

```
async_client = openai.AsyncOpenAI()

async def acall_chat_model(messages: List[dict]) -> str:
    response = await async_client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=messages,
    )
    return response.choices[0].message.content

async def ainvoke_chain(topic: str) -> str:
    prompt_value = prompt_template.format(topic=topic)
    messages = [{"role": "user", "content": prompt_value}]
    return await acall_chat_model(messages)

await ainvoke_chain("ice cream")
```

LCEL

```
await chain.ainvoke("ice cream")
```

LLM instead of chat model

If we want to use a completion endpoint instead of a chat endpoint:

Without LCEL

```
def call_llm(prompt_value: str) -> str:
    response = client.completions.create(
        model="gpt-3.5-turbo-instruct",
        prompt=prompt_value,
    )
    return response.choices[0].text

def invoke_llm_chain(topic: str) -> str:
    prompt_value = prompt_template.format(topic=topic)
    return call_llm(prompt_value)

invoke_llm_chain("ice cream")
```

LCEL

```
from langchain_openai import OpenAI

llm = OpenAI(model="gpt-3.5-turbo-instruct")
llm_chain = (
    {"topic": RunnablePassthrough()}
    | prompt
    | llm
    | output_parser
)

llm_chain.invoke("ice cream")
```

Different model provider

If we want to use Anthropic instead of OpenAI:

Without LCEL

```
import anthropic

anthropic_template = f"Human:\n\n{prompt_template}\n\nAssista
anthropic_client = anthropic.Anthropic()

def call_anthropic(prompt_value: str) -> str:
    response = anthropic_client.completions.create(
        model="claude-2",
        prompt=prompt_value,
        max_tokens_to_sample=256,
    )
    return response.completion

def invoke_anthropic_chain(topic: str) -> str:
    prompt_value = anthropic_template.format(topic=topic)
    return call_anthropic(prompt_value)

invoke_anthropic_chain("ice cream")
```

LCEL

```
from langchain_anthropic import ChatAnthropic

anthropic = ChatAnthropic(model="claude-2")
anthropic_chain = (
    {"topic": RunnablePassthrough()}
    | prompt
    | anthropic
    | output_parser
)

anthropic_chain.invoke("ice cream")
```

Fallbacks

If we wanted to add fallback logic, in case one model API is down:

Without LCEL

```
def invoke_chain_with_fallback(topic: str) -> str:
    try:
        return invoke_chain(topic)
    except Exception:
        return invoke_anthropic_chain(topic)

async def ainvoke_chain_with_fallback(topic: str) -> str:
    try:
        return await ainvoke_chain(topic)
    except Exception:
        # Note: we haven't actually implemented this.
        return await ainvoke_anthropic_chain(topic)

async def batch_chain_with_fallback(topics: List[str]) -> str:
    try:
        return batch_chain(topics)
    except Exception:
        # Note: we haven't actually implemented this.
        return batch_anthropic_chain(topics)

invoke_chain_with_fallback("ice cream")
# await ainvoke_chain_with_fallback("ice cream")
batch_chain_with_fallback(["ice cream", "spaghetti", "dumplings"])
```

LCEL

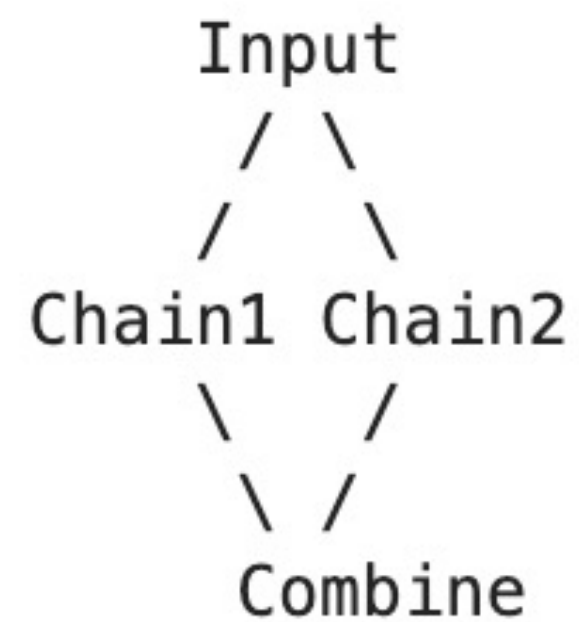
```
fallback_chain = chain.with_fallbacks([anthropic_chain])

fallback_chain.invoke("ice cream")
# await fallback_chain.ainvoke("ice cream")
fallback_chain.batch(["ice cream", "spaghetti", "dumplings"])
```

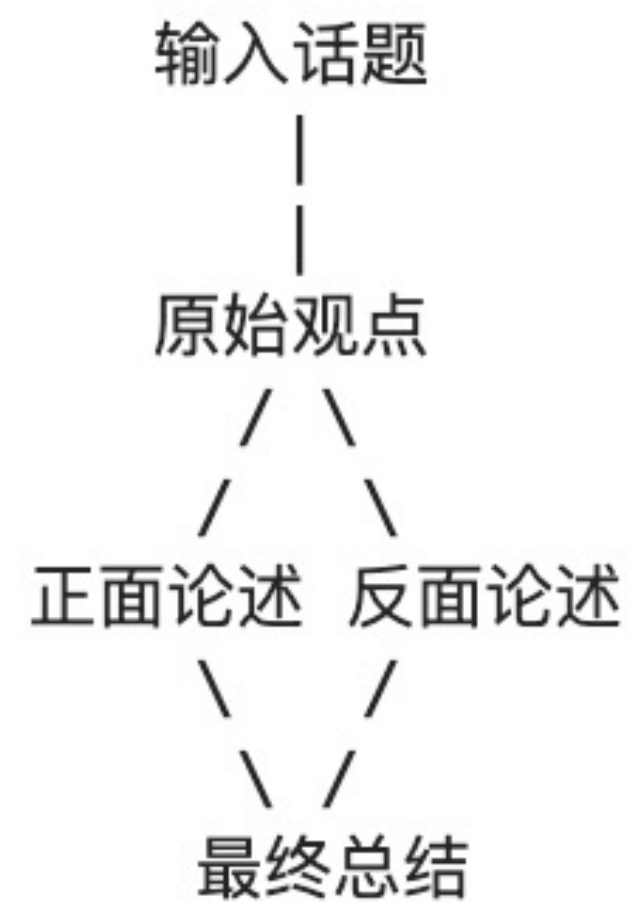
LCEL 进阶使用：整合复杂逻辑的多链

Multiple Chain 快速入门

Runnables 可以轻松地被用来串联多个 Chains，使用 RunnablePassthrough 将输出同时传给多条后继链。



本指南展示如何使用 Runnable 实现多个 AI 关于相同话题的辩论：



```

# 导入相关模块, 包括运算符、输出解析器、聊天模板、ChatOpenAI 和 运行器
from operator import itemgetter
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.runnables import RunnablePassthrough

# 创建一个计划器, 生成一个关于给定输入的论证
planner = (
    ChatPromptTemplate.from_template("生成关于以下内容的论点: {input}")
    | ChatOpenAI()
    | StrOutputParser()
    | {"base_response": RunnablePassthrough()}
)

# 创建正面论证的处理链, 列出关于基础回应的正面或有利的方面
arguments_for = (
    ChatPromptTemplate.from_template(
        "列出关于{base_response}的正面或有利的方面"
    )
    | ChatOpenAI()
    | StrOutputParser()
)

# 创建反面论证的处理链, 列出关于基础回应的反面或不利的方面
arguments_against = (
    ChatPromptTemplate.from_template(
        "列出关于{base_response}的反面或不利的方面"
    )
    | ChatOpenAI()
    | StrOutputParser()
)

```

```

# 创建最终响应者, 综合原始回应和正反论点生成最终的回应
final_responder = (
    ChatPromptTemplate.from_messages(
        [
            ("ai", "{original_response}"),
            ("human", "正面观点:\n{results_1}\n\n反面观点:\n{results_2}"),
            ("system", "给出批评后生成最终回应"),
        ]
    )
    | ChatOpenAI()
    | StrOutputParser()
)

# 构建完整的处理链, 从生成论点到列出正反论点, 再到生成最终回应
chain = (
    planner
    | {
        "results_1": arguments_for,
        "results_2": arguments_against,
        "original_response": itemgetter("base_response"),
    }
    | final_responder
)

```

```
chain.invoke({"input": "房地产低迷"})
```

'在面对房地产低迷的情况下, 我们需要综合考虑正面和反面观点, 以找到平衡和解决问题的方法。和市场自我调整, 来应对潜在的风险和挑战。同时, 房地产市场的低迷也可能为购房者提供更好的价格并寻求合适的解决方案, 以实现经济的平稳发展和社会的和谐进步。'

流式输出

```

## chain 最终输出经过了 StrOutputParser 处理, 所以可以直接输出流式输出 s
for s in chain.stream({"input": "全球经济"}):
    print(s, end="", flush=True)

```

尽管国际合作和协调在推动全球经济稳定和可持续发展方面可能会遇到一些困难和挑战, 但我们仍须坚持合作和可持续发展。国际合作不仅可以促进经济增长和贸易自由化, 还能够解决全球性问题, 为全球经济挑战。

实战 **LCEL** 开发与应用

DEMO

THANKS

 极客时间 | 训练营